

Capítulo 5

Mais SQL: Consultas Complexas, Triggers, Views e Modificação de Esquema

Tópicos

- Consultas de Recuperação SQL mais Complexas
- Especificando Restrições Como Asserções e Ações Como Triggers
- Visões (Views) – Tabelas Virtuais em SQL
- Instruções de Alteração de Esquema em SQL

Consultas de recuperação SQL mais complexas

- Recursos adicionais que permitem que os usuários especifiquem recuperações mais complexas do banco de dados:
 - Consultas aninhadas, tabelas de junção, junções externas, funções agregadas e agrupamento

Comparações envolvendo NULL e lógica de três valores

- Significados de NULL
 - **Valor desconhecido**
 - **Valor indisponível ou retido**
 - **Atributo não aplicável**
- Cada valor NULL individual é considerado diferente de qualquer outro valor NULL
- SQL usa uma lógica de três valores:
 - TRUE, FALSE e UNKNOWN

Comparações envolvendo NULL e lógica de três valores (cont.)

Tabela 5.1

Conectivos lógicos na lógica de três valores.

(a) AND	TRUE	FALSE	UNKNOWN
	TRUE	TRUE	FALSE
	FALSE	FALSE	FALSE
	UNKNOWN	UNKNOWN	FALSE
(b) OR	TRUE	FALSE	UNKNOWN
	TRUE	TRUE	TRUE
	FALSE	TRUE	FALSE
	UNKNOWN	TRUE	UNKNOWN
(c) NOT			
	TRUE	FALSE	
	FALSE	TRUE	
	UNKNOWN	UNKNOWN	

Comparações envolvendo NULL e lógica de três valores (cont.)

- A SQL permite consultas que verificam se o valor de um atributo é NULL
 - IS ou IS NOT NULL

Consulta 18. Recuperar os nomes de todos os funcionários que não possuem supervisores.

```
C18:  SELECT  Pnome, Unome  
      FROM    FUNCIONARIO  
      WHERE   Cpf_supervisor IS NULL;
```

Consultas aninhadas, tuplas e comparações de conjunto/multiconjunto

■ Consultas aninhadas

- São blocos select-from-where completos dentro da cláusula WHERE de outra consulta
- **Consulta externa**

■ Operador de comparação IN

- Compara um valor v com um conjunto (ou multiconjunto) de valores V
- Avalia como TRUE se v for um dos elementos em V

Consultas aninhadas (cont.)

```
C4A: SELECT DISTINCT Projnumero  
FROM PROJETO  
WHERE Projnumero IN  
    ( SELECT Projnumero  
      FROM PROJETO,  
            DEPARTAMENTO,  
            FUNCIONARIO  
      Dnum=Dnumero AND  
      WHERE Cpf_gerente=Cpf  
            AND Unome='Silva' )  
OR Projnumero IN  
    ( SELECT Pnr  
      FROM TRABALHA_EM,  
            FUNCIONARIO  
      WHERE Fcpf=Cpf AND  
            Unome='Silva' );
```

Consultas aninhadas (cont.)

- Permite o uso de tuplas de valores em comparações
 - Colocando-os entre parênteses

```
SELECT DISTINCT Fcpf
FROM   TRABALHA_EM
WHERE  (Pnr, Horas) IN ( SELECT Pnr, Horas
                        FROM   TRABALHA_EM
                        WHERE   Fcpf='12345678966' );
```

Consultas aninhadas (cont.)

- Outros operadores de comparação podem ser usados para comparar um único valor v
 - O operador = ANY (ou = SOME)
 - Retorna TRUE se o valor v for igual a *algum valor* no conjunto V e portanto é equivalente a IN
 - Outros operadores que podem ser combinados com ANY (ou SOME): >, >=, <, <= e <>

```
C6: SELECT Pnome, Unome
      FROM  FUNCIONARIO
      WHERE NOT EXISTS ( SELECT*
                        FROM  DEPENDENTE
                        WHERE Cpf=Fcpf );
```

Consultas aninhadas(cont.)

- Para evitar erros e ambiguidades em potencial
 - Criar variáveis de tupla (apelidos) para todas as tabelas referenciadas em uma consulta SQL

Consulta 16. Recuperar o nome de cada funcionário que tem um dependente com o mesmo nome e com o mesmo sexo do funcionário.

```
C16: SELECT F.Pnome, F.Unome
      FROM  FUNCIONARIO AS F
      WHERE F.Cpf IN ( SELECT  D.Fcpf
                       FROM    DEPENDENTE
                           AS D
                       WHERE F.Pnome=
                           D.Nome_
                           dependente
                           AND
                           F.Sexo=
                           D.Sexo );
```

Consultas aninhadas correlacionadas

- Consultas aninhadas **correlacionadas**
 - É avaliada uma vez para cada tupla (ou combinação de tuplas) na consulta externa

As funções EXISTS e UNIQUE em SQL

- Função EXISTS
 - Verificar se o resultado de uma consulta aninhada correlacionada é vazio ou não
- EXISTS e NOT EXISTS
 - Costumam ser usados em conjunto com uma consulta aninhada correlacionada
- Função SQL UNIQUE(C)
 - Retorna TRUE se não houver tuplas duplicadas no resultado da consulta C

Conjuntos explícitos e renomeação de atributos em SQL

- É possível usar um conjunto explícito de valores na cláusula WHERE
- Usar o qualificador AS seguido pelo novo nome desejado
 - Renomear qualquer atributo que apareça no resultado de uma consulta

```
C8A: SELECT  F.Unome AS Nome_funcionario,  
            S.Unome AS Nome_supervisor  
FROM    FUNCIONARIO AS F,  
        FUNCIONARIO AS S  
WHERE   F.Cpf_supervisor=S.Cpf;
```

Tabelas de junção em SQL e junções externas (outer joins)

- **Tabela de junção**

- Permite aos usuários especificar uma tabela resultante de uma operação de junção na cláusula FROM de uma consulta

- A cláusula FROM em C1A

- Contém uma única tabela de junção

```
C1A: SELECT  Pnome, Unome, Endereco  
          FROM  (FUNCIONARIO JOIN  
                DEPARTAMENTO  
                ON Dnr=Dnumero)  
          WHERE Dnome='Pesquisa';
```

Tabelas de junção em SQL e junções externas (cont.)

- Especifique diferentes tipos de junção
 - NATURAL JOIN
 - Vários tipos de OUTER JOIN
- NATURAL JOIN sobre duas relações R e S
 - Nenhuma condição de junção é especificada
 - Condição EQUIJOIN implícita para cada par de atributos com o mesmo nome de R e S

Tabelas de junção em SQL e junções externas (cont.)

■ Inner join

- O tipo padrão de junção em uma tabela de junção
- Tupla é incluída no resultado somente se uma tupla combinar na outra relação

■ LEFT OUTER JOIN

- Toda tupla na tabela da esquerda precisa aparecer no resultado;
- Se não tiver uma tupla combinando
 - Preenchida com valores NULL para os atributos da tabela da direita

Tabelas de junção em SQL e junções externas (cont.)

- RIGHT OUTER JOIN

- Toda tupla na tabela da direita precisa aparecer no resultado
- Se não tiver uma tupla combinando
 - Preenchida com valores NULL para os atributos da tabela da esquerda

- FULL OUTER JOIN

- É possível aninhar especificações de junção

Funções de agregação em SQL

- São usadas para resumir informações de várias tuplas em uma síntese de tupla única
- **Agrupamento**
 - Cria subgrupos de tuplas antes do resumo
- Funções de agregação embutidas
 - **COUNT, SUM, MAX, MIN e AVG**
- Essas funções podem ser usadas na cláusula SELECT ou em uma cláusula HAVING

Funções de agregação em SQL (cont.)

- Valores NULL são descartados quando as funções de agregação são aplicadas a determinada coluna (atributo)

Consulta 20. Achar a soma dos salários de todos os funcionários do departamento 'Pesquisa', bem como o salário máximo, o salário mínimo e a média dos salários nesse departamento.

```
C20: SELECT SUM (Salario), MAX (Salario),  
           MIN (Salario), AVG (Salario)  
FROM      (FUNCIONARIO JOIN  
           DEPARTAMENTO ON Dnr=Dnumero)  
WHERE     Dnome='Pesquisa';
```

Consultas 21 e 22. Recuperar o número total de funcionários na empresa (C21) e o número de funcionários no departamento 'Pesquisa' (C22).

```
C21: SELECT COUNT (*)  
      FROM   FUNCIONARIO;  
C22: SELECT COUNT (*)  
      FROM   FUNCIONARIO, DEPARTAMENTO  
      WHERE  Dnr=Dnumero  
            AND Dnome='Pesquisa';
```

Agrupamento: as cláusulas **GROUP BY** e **HAVING**

- **Particionar** a relação em subconjunto de tuplas
 - Baseado no **atributo(s) de agrupamento**
 - Aplicar a função a cada grupo desse tipo independentemente
- Cláusula **GROUP BY**
 - Especifica os atributos de agrupamento

Exemplo

- `select dnr, count(*), avg(salario) from funcionario group by dnr;`
- `select dnr, count(*), max(salario) from funcionario group by dnr;`

Agrupamento: as cláusulas **GROUP BY** e **HAVING** (cont.)

- Se houver NULLs no atributo de agrupamento
 - Um grupo separado é criado para todas as tuplas com um valor NULL no atributo de agrupamento.

Grouping: The GROUP BY and HAVING Clauses (cont'd.)

- Cláusula **HAVING**

- Oferece uma condição sobre a informação de resumo

```
C28: SELECT Dnumero, COUNT (*)  
      FROM  DEPARTAMENTO, FUNCIONARIO  
      WHERE Dnumero=Dnr AND Salario>40.000 AND  
            ( SELECT Dnr IN  
              FROM  FUNCIONARIO  
              GROUP BY Dnr  
              HAVING COUNT (*) > 5)
```

Discussão e resumo das consultas em SQL

```
SELECT <lista atributo e função>  
FROM <lista tabela>  
[ WHERE <condição> ]  
[ GROUP BY <atributo(s) de agrupamento> ]  
[ HAVING <condição de grupo> ]  
[ ORDER BY <lista atributos> ];
```

Exemplo

- `SELECT max(salario),min(salario), count(*)
FROM (empresa.funcionario join
empresa.departamento on dnr =
dnumero)where dnome = 'Research';`
- #Para cada departamento, recuperar o número do departamento, o número de funcionários;# no departamento e seu salário médio.

Especificando restrições como asserções e ações como triggers

- **CREATE ASSERTION**

- Especifica tipos adicionais de restrições que estão fora do escopo das restrições embutidas do modelo relacional

- **CREATE TRIGGER**

- Especifica ações automáticas que o sistema de banco de dados realizará quando certos eventos e condições ocorrerem

Especificando restrições gerais como asserções em SQL

■ CREATE ASSERTION

- Especificar uma consulta que seleciona quaisquer tuplas que violam a condição desejada
- Usado somente em casos onde não é possível usar CHECK sobre atributos e domínios

```
CREATE ASSERTION RESTRICAO_SALARIAL
CHECK ( NOT EXISTS
      ( SELECT      *
        FROM        FUNCIONARIO F,
                     FUNCIONARIO G,
                     DEPARTAMENTO D
        WHERE        F.Salario>G.Salario
                     AND F.Dnr=D.Dnumero
                     AND D. Cpf_gerente =G.Cpf ) );
```

Introdução a Triggers em SQL

- Comando CREATE TRIGGER
 - Usada para monitorar o banco de dados
- Um trigger típico tem três componentes:
 - **Evento(s)**
 - **Condição**
 - **Ação**

Visões (views) – Tabelas virtuais em SQL

- Conceito de uma view em SQL
 - Tabela única derivada de outras tabelas
 - Considerada uma tabela virtual

Especificação das views em SQL

■ Comando **CREATE VIEW**

- Recebe um nome de tabela (virtual), ou nome de view, uma lista de nomes de atributo e uma consulta para especificar o conteúdo da view

```
V1: CREATE VIEW TRABALHA_EM1
    AS SELECT    Pnome, Unome, Projnome,
                Horas
    FROM          FUNCIONARIO, PROJETO,
                TRABALHA_EM
    WHERE         Cpf=Fcpf AND Pnr=Projnumero;

V2: CREATE VIEW DEP_INFO(Dep_nome, Qtd_
    func, Total_sal)
    AS SELECT    Dnome, COUNT (*), SUM
                (Salario)
    FROM          DEPARTAMENTO, FUNCIONARIO
    WHERE         Dnumero=Dnr
    GROUP BY     Dnome;
```

Especificação das views em SQL (cont.)

- Especificar consultas SQL em uma view
- View sempre atualizada
 - Responsabilidade do SGBD, e não do usuário
- Comando **DROP VIEW**
 - Descarta uma view

Implementação e atualização de view e views em linha

- É muito complexo implementar uma view de forma eficiente para consulta
- Estratégia de **modificação de consulta**
 - Modificar a consulta da view em uma consulta nas tabelas da base
 - Desvantagem: ineficaz para views definidas por consultas complexas que são demoradas para executar

Implementação da view

- Estratégia de **materialização de view**
 - Criar fisicamente uma tabela de view temporária quando a view for consultada pela primeira vez
 - Manter essa tabela na suposição de que outras consultas a view acontecerão em seguida
 - Requer estratégia eficiente para atualizar automaticamente a tabela da view quando as tabelas de base forem atualizadas

Implementação da view (cont.)

- Estratégia de **atualização incremental**
 - SGBD determina quais novas tuplas devem ser inseridas, excluídas ou modificadas em uma tabela de view materializada

Atualização de view e views em linha

- Atualização em uma view definida sobre uma única tabela sem quaisquer funções de agregação
 - Pode ser mapeada para uma atualização sobre a tabela da base
- View que envolve junções (joins)
 - Não é possível que o SGBD determine qual das atualizações é intencionada

Atualização de view e views em linha (cont.)

- Cláusula **WITH CHECK OPTION**
 - Precisa ser acrescentada ao final da definição de view se uma view tiver de ser atualizada
- **View em linha**
 - Definida na cláusula FROM de uma consulta em SQL

Instruções de alteração de esquema em SQL

- **Comandos de evolução de esquema**
 - Pode ser feito enquanto o banco de dados está operando
 - Não exige recompilação do esquema

O comando DROP

- Comando DROP
 - Usado para remover elementos nomeados do esquema, como tabelas, domínios ou restrições
- Opções de comportamento de drop:
 - CASCADE e RESTRICT
- Exemplo:
 - `DROP SCHEMA EMPRESA CASCADE;`

O comando ALTER

- **Ações de alteração de tabela** incluem:
 - Acrescentar ou remover uma coluna (atributo)
 - Alterar uma definição de coluna
 - Acrescentar ou remover restrições de tabela
- Exemplo:
 - ALTER TABLE EMPRESA.FUNCIONARIO ADD COLUMN Tarefa VARCHAR(12);
- Para remover uma coluna
 - Escolher CASCADE ou RESTRICT

O comando ALTER (cont.)

- Alterar as restrições especificadas sobre uma tabela
 - Acrescentar ou remover uma restrição nomeada

```
ALTER TABLE EMPRESA.FUNCIONARIO  
DROP CONSTRAINT CHESUPERFUNC CAS-  
CADE;
```

Resumo

- SQL complexo:
 - Consultas aninhadas, tabelas com junções, junções externas, funções de agregação e agrupamento
- CREATE ASSERTION e CREATE TRIGGER
- Views
 - Tabelas virtuais ou derivadas

